



PixieDuster CLI

Architecture and Implementation

Your Fairy Prompt-Mother, at the command line

4,435

lines of module code

412

tests, all offline

22

secret rules

0

network calls in mining or safety

Start here

In one sentence. It reads the writing you have already done in a project - your commit messages, your README, the notes you leave in your code - and works out how you write. Then it writes that down as instructions an AI can follow, so the AI sounds like you instead of sounding like a robot.

Why you would want it

Your web app does this already, but you have to find writing samples and upload them. This version skips that. Your repository is already full of your writing, so it just reads it.

How to use it

There is really only one command. Type this inside any project folder:

```
pixieduster clone
```

It then walks you through everything. You will see a gold PixieDuster logo, it will read the project, ask you three questions you answer with the arrow keys, and let you set how funny the persona should be with a slider. At the end it saves a file called `AGENTS.md` and shows you a certificate.

Two others, once you have that file:

```
pixieduster chat          # talk to it, hear how it sounds
pixieduster diff draft.md # ask: does this sound like me?
```

The one thing to be careful about

Your writing gets sent to Google to be analyzed. That is how it works, and there is no way around it. So before it sends anything, you can look at exactly what would go:

```
pixieduster clone --dry-run
```

That prints the full list and sends nothing at all. It also checks for passwords and keys hiding in your text and refuses to send quietly if it finds any.

IF YOU READ NOTHING ELSE

Run `pixieduster clone --dry-run` first to see what it would send. Then run `pixieduster clone` and answer the questions. Everything after this page is detail you only need when something breaks.

Contents

1 What it is

The same brain as the web app, fed from a git repo

2 The pipeline

Nine steps from a repo to an AGENTS.md

3 Module map

Eight modules, who owns what, who calls whom

4 Mining

Four sources, and why each is filtered the way it is

5 Safety

What stands between a private repo and Google

6 API key handling

Bring your own key, and why it has to be that way

7 The TUI

Rich, one Live region, and two bugs worth remembering

8 Tech stack

Everything the wheel depends on

9 Open items

What is resolved, and what to keep in mind

ABOUT THIS DOCUMENT

Every technical claim here was checked against the code in `pixieduster/` on 24 August 2026, and the measurements were re-run rather than quoted. Where the code and the brief disagreed, the code won.

1 What it is

PixieDuster clones a writing voice. The Streamlit app you built does it from uploaded samples: you hand it essays, it profiles the author behind them, and it hands back a persona prompt. The CLI is the same brain with a different mouth and a different diet. It takes a git repo instead of an upload, and it writes **AGENTS.md** or **CLAUDE.md** instead of a download button, so the coding agent working in that repo writes prose in your voice rather than generic LLM house style.

The insight it runs on is that most developers have already written a great deal in their own voice, and it is sitting in the repo: commit bodies, the README, docstrings, PR descriptions. Nobody has to upload anything. The material is already there, and it is more honest than an essay written to be sampled.

Web app and CLI, side by side

	Streamlit app (app.py)	PixieDuster CLI
Input	Uploaded files (PDF, DOCX, TXT)	A git repo: commits, docs, comments, PRs
Interface	Browser, HF Spaces	Terminal, Rich TUI
Interview	Multiple-choice, in-page	Arrow-key select, in-terminal
Output	A downloadable persona document	AGENTS.md / CLAUDE.md written into the repo
Shared	prompts.py and core.py	prompts.py and core.py
Distribution	A hosted Space	uvx pixieduster clone, no install

The sharing is real, not aspirational. `app.py` was rewired to import from `pixieduster.core` and `pixieduster.prompts`, so the rubric, the anti-AI template, the Benign Violation humor block and the Gemini call itself exist in exactly one place. A fix to the persona rubric lands in both products at once.

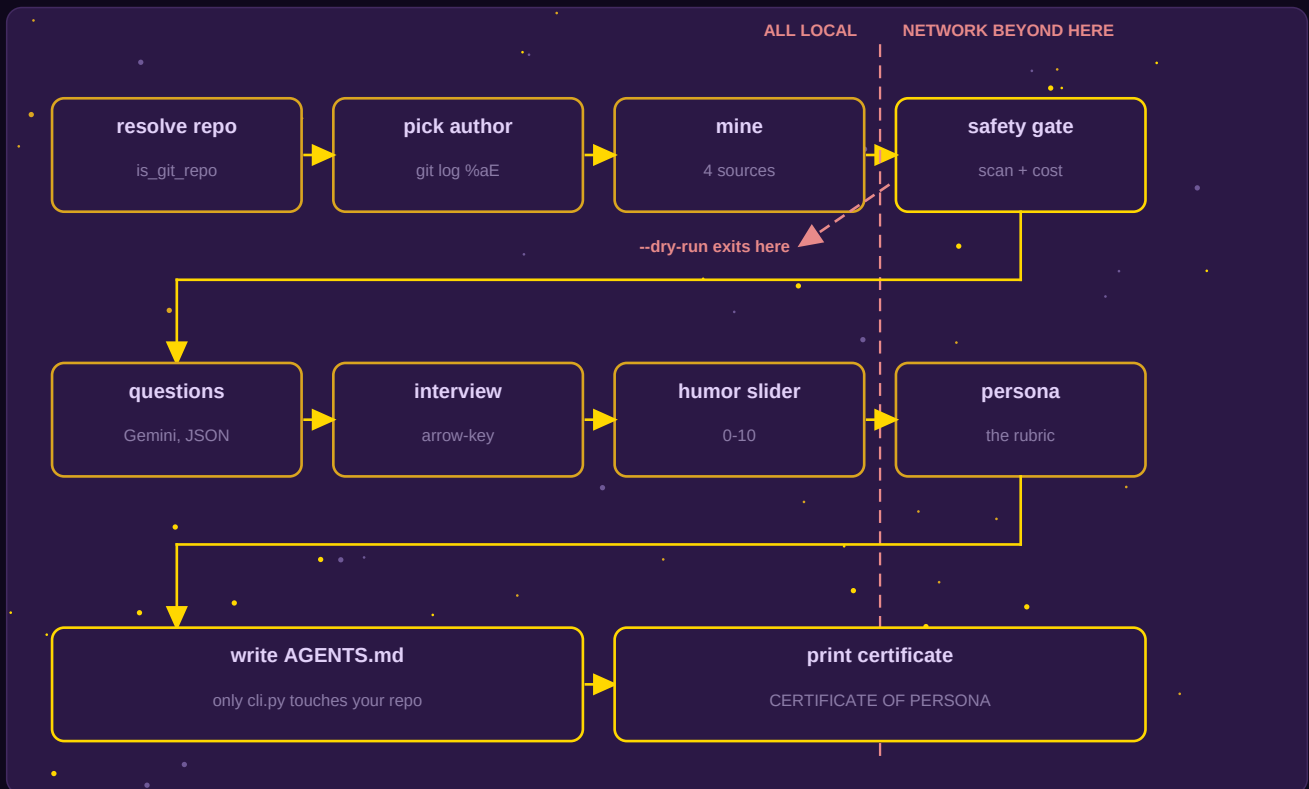
The four commands

Command	What it does
clone	Mine a repo, interview you, write the persona file. The main event.
chat	A REPL against a persona file, so you can hear how it sounds before trusting it.
diff <file>	Score a draft against the persona and list where the voice drifts.
config	set-key, show, models, set-model. Key display is always masked.

Both `pixieduster` and the shorter `pxd` are installed as entry points, and a fifth command, `version`, prints the version.

2 The pipeline

clone runs nine steps. Everything before the safety gate is local; nothing has left the machine yet. The gate is the only door out, and `--dry-run` ends the run right in front of it.



clone: nine steps. The dashed rule is the trust boundary; the gate is the only crossing.

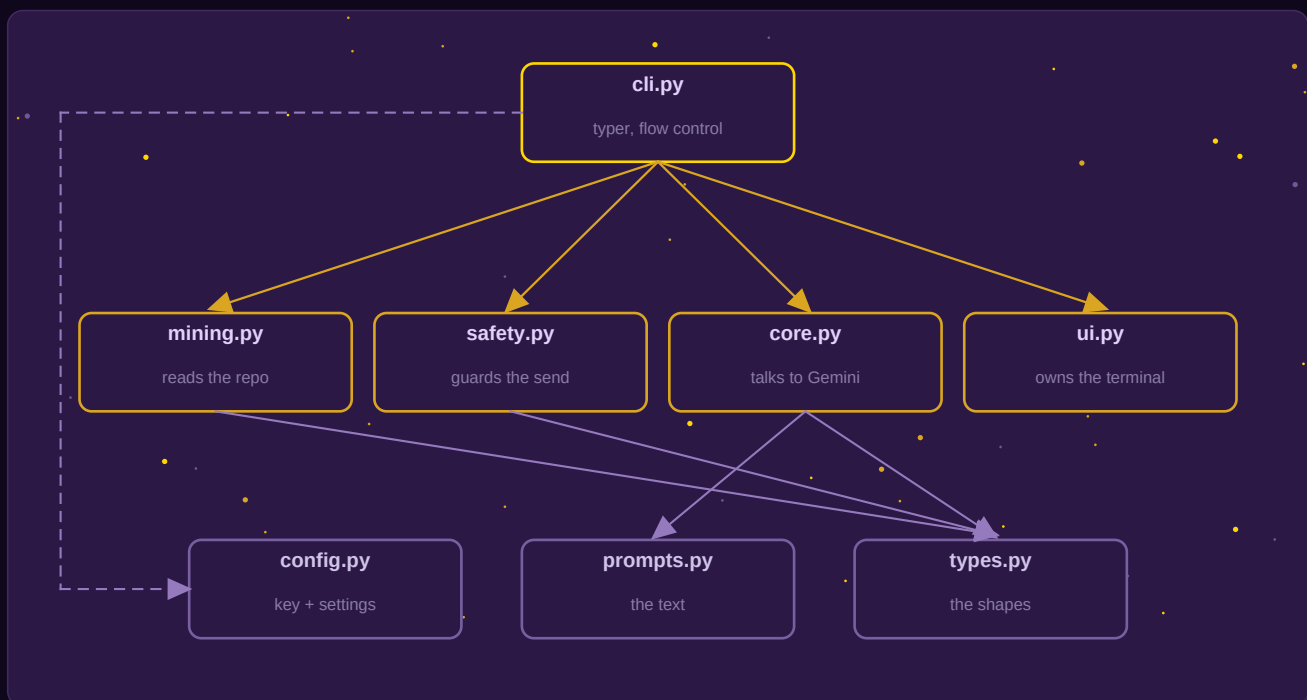
What each gate can stop

- **No samples.** A repo of one-line 'fix' commits has nothing to clone, and the CLI says so and exits rather than sending an empty prompt.
- **No key.** If nothing resolves a key it offers a one-time save, and refuses non-interactively with exit code 2.
- **Model not available.** `_verify_model()` checks the id against the live model list before anything expensive happens.
- **Findings.** Any secret found means an explicit confirmation, a redaction, or a stop.
- **Output exists.** An existing AGENTS.md is never silently overwritten; declining writes AGENTS-pixiedust.md instead.

3 Module map

Eight modules, 4,435 lines, plus 2,676 lines of tests. The shape is deliberate: one module knows about the network, one module knows about the filesystem of your repo, one module knows about your terminal, and none of them knows about the others.

Module	Lines	Job
types.py	53	Three dataclasses: Sample, Question, Finding. Imports nothing, so everyone can import it.
prompts.py	139	The rubric, the anti-AI template, the humor block, the question schema, the AGENTS.md header, the diff instruction. Text only.
config.py	280	Key resolution and the TOML config file. Hand-rolled TOML writing, because tomlib is read-only.
core.py	462	Everything that speaks to Gemini: call, chat, list models, generate questions, generate persona.
cli.py	467	Typer commands, the flow control, and the only writes to your repo.
safety.py	795	Secret detection, redaction, token and cost estimates, the dry-run report. Entirely offline.
ui.py	1061	Rich presentation: banner, dust, stages, arrow-key select, slider, certificate, tables.
mining.py	1171	Four miners, author attribution via blame, and the budget balancer.



Dependencies point downward only. Nothing in the middle row imports anything else in it.

Two invariants worth knowing

- **cli.py is the only module that writes to your repo.** Every other module reads, computes, or prints. If a file appeared in your working tree, one line in cli.py put it there.
- **mining.py and safety.py make no network calls at all.** mining.py shells out to git, and optionally to gh, which is the only subprocess with a socket behind it and is skipped entirely by --no-pr. safety.py touches nothing but strings.

4 Mining

Four sources, each with its own idea of what counts as your writing. All of them run over `git ls-files`, which means `.gitignore` is respected for free and no path outside the repository root is ever read.

Source	How	Share
commits	<code>git log --format=%B</code> : full bodies, not subjects. Merges, bots, dependency bumps and version noise dropped; anything under 40 characters after cleaning dropped.	40%
docs	Tracked <code>.md</code> / <code>.rst</code> / <code>.txt</code> . README, CONTRIBUTING and docs/** first. Code fences, badges, link-reference blocks and HTML stripped; CHANGELOG and LICENSE skipped.	25%
comments	Docstrings via <code>ast</code> , plus comment runs of two or more lines. Licence headers, shebangs, pragmas, type stubs and generated files skipped.	20%
PRs	<code>gh pr list --json</code> for bodies and review comments. If <code>gh</code> is missing or unauthenticated it returns an empty list silently, and never blocks.	15%

Why commit bodies, not subjects

A subject line is a label. A body is prose: it argues, it hedges, it explains why. That is where a voice lives, so `mine_commits` reads full bodies and throws away anything too short to be prose.

Stripping the co-author

Commit messages written with a coding agent carry trailers: Co-Authored-By, Claude-Session, and a 'Generated with Claude Code' line. Left in, they are the loudest, most repetitive text in the corpus, and the model would dutifully learn them. You would clone Claude instead of yourself. So they are stripped, along with the usual Signed-off-by / Reviewed-by family and a generic `<Something>-by`: trailer pattern.

Comments via tokenize, not regex

This one is your repo's fault, and it is a good bug. `app.py` contains a triple-quoted prompt string with `## Core Directives` inside it. A naive line scan for lines starting with `#` would harvest that as your own prose - you would be cloning the voice of a prompt you wrote for the model, not the voice you write in. Running the source through `tokenize` means only real COMMENT tokens are seen, and string contents are invisible by construction. The naive scanner survives only as a fallback for files that do not tokenize.

PIXIEDUSTER/MINING.PY

```
# _python_comment_runs: only real COMMENT tokens count
for tok in tokenize.generate_tokens(io.StringIO(source).readline):
    if tok.type != tokenize.COMMENT:
        continue
    row, col = tok.start
    if lines[row - 1][:col].strip():
        continue          # trailing comment on a code line, not prose
```

Sharing out a limited budget

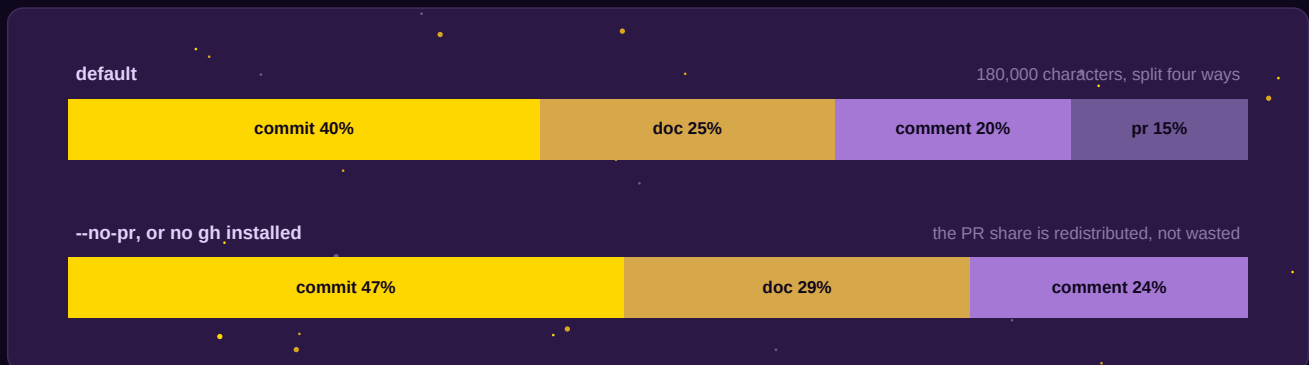
You cannot send a whole repository to the AI - too much text, and you pay for every word of it. So `mine_all` takes about 180,000 characters, roughly 45 pages, and stops.

The catch is which 45 pages. If it simply took the first 180,000 characters and you had 400 commits, the commits would eat all of it, and the AI would only ever see commit messages - short, technical, typed in a hurry. You would

get a persona that sounds like a changelog. So room is reserved in advance: 40% commits, 25% docs, 20% code comments, 15% pull requests. Four different moods of the same person.

Two adjustments then keep it from wasting space:

- **Leftovers get handed on.** If a group does not need all its space, the rest is split among the groups that are still short. This repository has no pull requests, so that 15% would sit empty - instead it goes to the commits and the README.
- **Biggest goes first.** If a group has too much, the longest items are dropped before the short ones. Five ordinary pieces show your range better than one enormous document.



A source that is absent or small hands its unused share to the groups that are still short.

Working out who wrote what

This only matters on a repository with several people on it. Git records who wrote each commit, so filtering those by author is easy. A README is different - it is just a file sitting there, with nothing on it saying who wrote which part. Left alone, you would get your commits blended with everybody's README.

So it uses `git blame`, which reports who wrote each individual line, and keeps only the target's lines. Blame is slow, so it is bounded: at most 40 files, and 15 seconds for the whole phase. A cheaper `git log --follow` pass runs first to drop files the person never touched at all, and if blame fails or runs out of time the code falls back to that cheaper answer instead of hanging. A block of comments has to be at least 60% theirs to count.

MEASURED ON THIS REPOSITORY

14 commits in history, 11 survive filtering. The three that were dropped - 'Fix NameError: add missing time import', 'Fix Pyodide import error for fpdf2' and 'Initial commit (Secrets redacted)' - were all under the 40-character floor, not noise-matched. Filtered to me@gretchenboria.com the full mine returns 13 samples: 11 commits, 1 doc, 1 comment, 0 PRs. Every commit in this repo is yours, so the author filter changes nothing here.

5 Safety

This is the section that matters. The whole point of the tool is that your repository's text goes to a third party. Google sees your commit bodies, your README and your docstrings. That is a reasonable trade for what you get back, but it is only reasonable if it is visible and if credentials do not go with it. `safety.py` is the last thing standing in that doorway, and it makes no network calls of any kind.

22 rules, priority-ordered

The rules are a list, not a set, and the order is load-bearing. Rules are tried in sequence and the first to claim a span of text wins, so a vendor-specific rule always beats a generic one and each secret is reported exactly once. A Google key inside `GOOGLE_KEY=AIza...` is reported as `google-api-key`, not as a generic `.env` value.

Rule	Shape	Sev
private-key-pem-block	The whole BEGIN/END block, so the body is scrubbed and not just the header. A second rule catches a truncated header-only paste.	high
aws-access-key-id	The fixed 4-char type prefixes (AKIA, ASIA, AROA, ...) plus 16 base32 chars.	high
aws-secret-access-key	40 base64 chars, but only next to an AWS-ish key name. Matching bare 40-char base64 would flag every git hash.	high
google-api-key	Alza + 35. This is the key the CLI itself uses, so it must never travel back out inside a sample.	high
anthropic-api-key	sk-ant-... Listed before the OpenAI rule.	high
openai-api-key	sk- with a negative lookahead for ant-, so Anthropic keys are not swallowed here.	high
github-token	ghp_ / gho_ / ghu_ / ghs_ / ghr_, plus github_pat_ for fine-grained PATs.	high
slack-token	xox[baprs]- prefixes, plus a separate rule for hooks.slack.com webhook URLs.	high
stripe-secret-key	sk_live_ and rk_live_ only. sk_test_ is not worth alarming over.	high
jwt	Two base64url segments that both start eyJ, plus a signature that may be empty for alg=none.	high
connection-string-credentials	postgres:// mysql:// mongodb+srv:// redis:// amqp:// and friends, with user:pass inline.	high
generic-secret-assignment	password / token / api_key / client_secret = <value>. The workhorse, and the noisy one.	med
dotenv-high-entropy-value	SCREAMING_SNAKE=<32+ chars of key-ish material> alone on a line.	med

Thirteen of the twenty-two are shown; the rest are the same idea applied to Google OAuth client secrets, npm, PyPI, SendGrid, Hugging Face and Authorization headers. Twenty are high severity, two are medium.

Keeping false positives survivable

The detector is biased toward recall, because a missed secret is a leak and a false positive is only an extra confirmation prompt. That bias is what makes the two generic rules noisy, so they are paired with structural filters that run after the regex matches:

- **Placeholder detection.** `your-key-here`, `changeme`, `xxxx`, `TODO` - matched as whole tokens inside the value, so a real key that happens to contain 'fake' as a substring is not excused.

- **Code-identifier shape.** `Optional[str]`, `get_password()`, `os.environ['X']`, `settings.SECRET_KEY`. `snake_case`, `camelCase`, `CONSTANT_CASE` and dotted attribute access are all rejected: a real credential essentially never has that shape.
- **Template syntax.** `${...}`, `{{...}}`, and anything starting with `<{ $ % & ! @ ?` is interpolation, not a literal.
- **A Shannon entropy floor.** 3.0 bits per character for a generic assignment, 3.5 for a `.env` value. English words and identifiers sit below that; random credential material sits above it.
- **Paths and URLs.** A value starting with `/` `.` `~` `http://` is a location, not a secret.

Measured, not asserted

BENCHMARKS

Zero false positives on a curated corpus of 39 innocent strings. Scanning 1,243 real `.py` files - 16.5 MB from the CLI's own `virtualenv` - produced exactly 3 findings: two generic-secret-assignment hits inside a numerical library's test data and one connection-string match in a URL-parsing test. All three are dismissible at a glance. The scan ran at roughly 5.4 MB/s with no catastrophic backtracking, which is what you want from a gate that runs before every send.

One pass, two outputs

`scan()` and `redact()` share a single `_collect()` pass. That is not an optimisation, it is a correctness property: if they had separate implementations they could disagree, and the interesting way for them to disagree is for the report to say 'clean' while the redactor leaves something in. One pass makes that impossible.

On top of that there is a belt-and-braces check. Every excerpt that goes into a Finding is run through `_leaks()`, which slides an 8-character window over the original secret and asks whether any distinctive run of it survived in the excerpt. If one did, the excerpt is thrown away and replaced with a bare marker. A Finding is safe to print, log, or paste into a GitHub issue, even if a rule elsewhere is buggy.



IF YOU EVER EXTEND THIS

SECRET_RULES is not the detector. The entropy floor, the placeholder list and the identifier-shape rejection live outside the rule list in a separate validators table, keyed by rule name. Code that reads `SECRET_RULES` and re-implements matching from the regexes gets the recall without any of the precision, and will flag `Optional[str]` as a credential. Always go through `scan()` and `redact()`.

The three ways out

- **--dry-run** prints the exact payload - every sample with its kind, origin and token count, the totals, the cost estimate and every finding - then exits. No network request is made. The report is plain text with no ANSI, so it pipes.
- **--scrub** replaces every match with <REDACTED:rule> and sends the redacted text instead of asking.
- **Confirmation.** A cost estimate and an explicit prompt precede every send. With findings present the prompt defaults to No. --yes skips it for scripted use.

THE PAYLOAD, BEFORE ANY OF IT LEAVES

```
$ pixieduster clone --dry-run
PIXIEDUSTER DRY RUN
=====
Nothing below has been sent anywhere. This is what *would* be sent.

SAMPLES (13)
  1. [commit] git log 0c7d3f1 (61 tokens)  author=me@gretchenboria.com

TOTALS
  estimated input tokens:  4812
  estimated cost (gemini-3.6-flash): $0.0036 (input only, estimate)

SECRET SCAN (0 finding(s))
  No potential secrets detected.
  NOTE: detection is best-effort. Review the sample list above.

END OF DRY RUN. No network request was made.
```

6 API key handling

PixieDuster is bring-your-own-key, and setup is a one-time thing. The key is resolved in a fixed order, first hit wins:



Falls through left to right. If nothing hits, clone offers a one-time save and then refuses.

Why the CLI cannot ship with a key

Two reasons, and neither has a workaround. First, a key embedded in a published CLI is not secret: the wheel is a zip file, and anyone who runs strings over it has your credential in about four seconds. Obfuscation only changes how long the four seconds take. Second, and worse, the spend is uncapped and lands on your card - every user of the tool, forever, billed to you, with no per-user limit you could enforce. BYOK is not a cop-out here; it is the only design that does not end in a surprise invoice.

Four specifics that are actually load-bearing

- **The key goes in a header, never a URL.** It is sent as `x-goog-api-key` rather than as a `?key=` query parameter. This is a direct fix to something the web app does: an API error that echoes the request URL back to you puts your key in the error text, in the traceback, and in whatever issue you paste it into.
- **Writes are atomic and never briefly world-readable.** `save_api_key` does `mkdir 0700`, then `os.open(..., 0o600)` to a temp file, then `os.replace`. There is no window in which the key exists at default permissions, and other settings already in the file are preserved rather than clobbered.
- **The key never enters `os.environ`.** `resolve_api_key` reads `.env` with `dotenv_values` rather than `load_dotenv`. `load_dotenv` would push the key into the process environment, where any subprocess inherits it and any crash dump captures it. Because the real environment is checked first anyway, this is behaviorally identical to `load_dotenv(override=False)` and strictly safer.
- **Errors are safe to paste.** `GeminiError` passes every message through a sanitizer that strips `?key=` and `&api_key=` parameters, redacts anything shaped like `AIza...`, and - when the live key is known - removes that exact string too. Three independent nets, because the interesting failure is the one you did not predict.

CONFIG SHOW NEVER PRINTS THE MIDDLE OF THE KEY

```
$ pixieduster config show
config file : /Users/you/.config/pixieduster/config.toml
key source  : dotenv
key         : AIza...4f21
model       : gemini-3.6-flash
```

The masking helper shows the first four and last four characters and nothing between, and for a key of eight characters or fewer it degrades to showing almost nothing rather than most of it.

7 The TUI

Rich and the standard library. No Textual, no readchar, no questionnaire. The whole terminal layer is one module with no dependency beyond what the CLI already needs, and it carries the web app's identity across: gold on deep purple, the same six processing stages, the same certificate.

- **A gradient ASCII wordmark.** PIXIEDUSTER rendered gold to purple, sized to fit 80 columns.
- **Falling pixie dust.** A Rich Live region updated at 10fps from a daemon thread, drifting the glyphs the web app uses. Transient, so it leaves no trace when it exits, and torn down cleanly on Ctrl-C.
- **A stage list.** The same six steps - inspecting samples, formulating questions, Big Five, LIWC, cognitive style, sociolinguistics - each with a spinner that ticks to a gold checkmark as the next one starts.
- **Arrow-key select and a gold slider.** Single-select for the interview questions, and a ---o--- bar for the humor level, both driven by raw key reads.
- **The certificate.** A double-bordered panel: CERTIFICATE OF PERSONA in letter-spaced gold, 'Officially cloned for: NAME', the rendered markdown body between gold rules, and 'Authorized by PixieDuster' at the foot. A direct port of the web app's certificate.

Two bugs worth remembering

1. BUFFERING ATE EVERY ARROW KEY

Reading raw keys with `sys.stdin.read(1)` silently broke all four arrow keys. An arrow arrives as an escape sequence - ESC, then [, then A - and Python's buffered TextIOWrapper swallows the tail, so the code saw a bare ESC and interpreted it as 'cancel'. Nothing errored; the arrows simply did nothing. The fix is a thin unbuffered wrapper around `os.read(fd, n)`, with `select` used to decide whether more of the sequence is pending. The decoder itself is a pure function, so it is tested without a TTY at all.

2. RICH HAS EXACTLY ONE LIVE REGION

`dust()` and `stages()` both need it, so they must never nest. The rule the code follows is simple: dust runs during chat, stages run during clone. That was a convention in a comment, which is how it would have come back. It is now enforced: each of the four widgets claims the region through a guard, and nesting any two raises `LiveConflictError` naming both, instead of silently corrupting the display. Eight tests cover it, including one that reads the source of clone, chat and diff and fails if any of them ever uses both.

Degrading properly

Decoration is suppressed when any of three conditions holds: `ui.PLAIN` is set by `--plain`, `NO_COLOR` is in the environment, or `stdout` is not a TTY. The third is what makes CI and piping work without anyone remembering a flag: piping the output produces zero ANSI bytes, the stage list becomes one plain line per step, arrow-key select becomes numbered input, and the certificate becomes a rule-and-center-text block.

8 Tech stack

Layer	Choice	Why
Language	Python >= 3.11	tomllib in the stdlib; matches the web app.
TUI	rich >= 13.7	Live regions, panels, tables, markdown rendering. No Textual.
CLI	typer >= 0.12	Type-hint-driven commands and flags.
HTTP	requests >= 2.31	Gemini REST v1beta, called directly.
Config	python-dotenv >= 1.0	Reading .env without polluting os.environ.
Optional (web)	streamlit, fpdf2	The Streamlit app and its PDF export, behind the 'web' extra.
Dev	pytest >= 8.0	412 tests, all offline.
Build	hatchling	The build backend declared in pyproject.toml.
Entry points	pixieduster, pxd	Both installed; pxd is the short alias.
Distribution	uvx pixieduster clone	Runs with no install at all.
State	one TOML file	~/.config/pixieduster/config.toml. Nothing else persists.

No SDK

Gemini is called over plain REST with requests rather than through google-genai. The API surface actually used is small - generateContent, and a GET of the model list - and vendoring an SDK for that would add a dependency tree, a release cadence and a set of transitive pins to a tool whose entire runtime footprint is currently four packages. Four dependencies is something a person can audit before pointing the thing at a private repo.

PIXIEDUSTER/CORE.PY

```
# what the whole API surface amounts to
POST {API_BASE}/models/{model}:generateContent
  headers: {'x-goog-api-key': key}
  body:    {'contents': [...], 'generationConfig': {...}}

GET {API_BASE}/models          # used only to verify the model id
```

Statelessness

There is no database, no cache directory, no history file and no telemetry. The one piece of persistent state is the TOML config: your key under [auth], and a small [settings] table holding your chosen model and the id of the last model verified against the live list. Deleting that file returns the tool to first-run.

9 Open items

Four things to know. The model-id question and the key problem are both resolved.

1. Resolved: the default model is real

`gemini-3.6-flash` was doubted during the build. It is genuine - confirmed against a live model list of 50 on 24 August 2026. No action needed. `_verify_model()` still checks the id on first use and falls back to the best available flash model if a future id goes stale, caching the answer so it costs one GET, once.

2. Resolved: two keys, one dead

A revoked key exported from `~/ .zshrc` was shadowing the working key in `.env`, because resolution checks the environment before `.env`. The dead export is now commented out and the working key verified. The general lesson survives the fix: when a key fails, check `config show` to see which of the four sources actually won.

A GEMINI KEY IS NOT ALWAYS AIZA

Both `AIza`-prefixed keys and longer `AQ.A`-prefixed keys are valid. Do not judge a key by its prefix.

3. Cost estimates are approximate, and sometimes absent

`estimate_cost` returns `None` rather than a guess for any unpriced model, so callers must render 'cost unknown' rather than `$0.00`. Output tokens cannot be known before the call, so the dry-run figure is input-only, and token counts are a `len/4` approximation - fine for budgeting, wrong for billing. The promotional `$0.75 / $3.75` rate for the 3.x flash models doubles after 31 December 2026, which the table does not model, so later estimates are a floor.

4. Secret detection is best-effort

It is pattern matching. It will not catch a credential with no recognizable shape, a secret split across lines, or something encoded before it was committed. The benchmarks in section 5 measure precision on real code, not recall against a determined leak. Review the `--dry-run` output before pointing this at a sensitive repository.

Run `clone --dry-run` once to see what your own repo gives up, and then let it write.